UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES EXTENDIDOS

Sistemas Distribuidos

Clase 07 – Logs distribuidos y procesamiento de eventos

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Segundo cuatrimestre 2026

Equipo docente

2026

1 Propósito y pregunta central

La clase 6 estudió cómo un motor de almacenamiento conserva una historia local mediante WAL, páginas, versiones e índices. Esta clase da el siguiente paso: utiliza una historia durable como *interfaz distribuida* entre productores y consumidores que pueden fallar, reiniciarse y avanzar a velocidades diferentes.

La pregunta central es:

> ¿Cómo convertir hechos producidos por una aplicación en una historia durable, ordenada por partes y reprocesable, sin confundir publicación, entrega, progreso y efecto de negocio?

Un log distribuido permite que el productor registre un hecho sin esperar que todos los destinatarios terminen su trabajo. Inventario, facturación, notificaciones y analítica pueden leer la misma historia con ritmos y objetivos distintos. Este desacoplamiento es valioso, pero introduce decisiones nuevas:

- qué hechos deben conservarse y durante cuánto tiempo;
- qué orden necesita el dominio;
- cómo se reparte el trabajo entre particiones y consumidores;
- qué significa que un consumidor haya "avanzado";
- qué ocurre si una falla separa el efecto de negocio del progreso registrado;
- qué parte de una promesa de *exactly-once* está realmente coordinada;
- cómo se procesan eventos cuyo tiempo de ocurrencia no coincide con su tiempo de llegada.

Al finalizar, el estudiante debería poder diseñar y defender un pipeline indicando el problema, la falla, la garantía, el mecanismo, el costo y la evidencia que permitiría validarlo.

2 Cómo estudiar este capítulo

La secuencia pedagógica coincide con la utilizada en la clase anterior:

$$\text{contexto} \to \text{intuición} \to \text{definición} \to \text{ejecución} \to \text{falla} \to \text{evidencia} \to \text{decisión}.$$

Para cada concepto conviene realizar cuatro acciones:

1. describir primero el comportamiento esperado sin nombrar una tecnología;
2. dibujar el camino normal y mover un crash o una pérdida de mensaje;
3. declarar exactamente qué estado participa en la garantía;
4. cerrar con una métrica o un experimento que pueda refutar la solución.

> Las analogías —película, carril, señalador o reloj del hecho— son ayudas iniciales. Una explicación completa debe abandonarlas y utilizar después el modelo técnico de topic, partición, offset, transacción y estado.

3 Base mínima para esta clase

Esta unidad no presupone conocer detalles avanzados de sistemas de archivos. Trabajaremos primero con siete objetos concretos:

Memoria de trabajo. Estado rápido que puede desaparecer al reiniciar el proceso.

Registro. Unidad individual que representa un hecho, por ejemplo "la compra 9F2A fue confirmada".

Append. Operación de agregar un registro al final de una secuencia sin reescribir la historia completa.

Log. Secuencia retenida de registros que puede volver a leerse.

Lector. Proceso que recorre esa secuencia y produce un resultado.

Posición actual. Punto alcanzado por la instancia viva; puede existir solo en memoria.

Progreso durable. Marca persistida desde la que una nueva instancia puede continuar después de una falla.

Considere el log:

$$[416, 417, 418, 419, 420].$$

Un consumidor alcanzó la posición 421, pero solo persistió progreso hasta 418. Si cae, la posición volátil desaparece y la recuperación utiliza la marca durable. Por eso parte del trabajo puede reaparecer.



También conviene comparar dos formas de integrar CampusShop:

- con llamadas sincrónicas, checkout espera inventario, correo y analítica, y hereda sus latencias y fallas;
- con un hecho durable, checkout espera la garantía de publicación y cada consumidor continúa luego con su propia política de error.

El log no elimina complejidad: la desplaza hacia orden, particiones, progreso, reintentos, idempotencia y observabilidad.

4 Contexto: del WAL local al log compartido

WAL y log distribuido comparten una idea: conservar una secuencia ordenada permite reconstruir trabajo después de una falla. Sin embargo, sus fronteras son distintas.

	WAL de un motor	Log distribuido
Propósito principal	recuperar el estado de un nodo	integrar productores y consumidores
Lectores	recovery del motor	aplicaciones independientes
Posición	LSN u otra secuencia interna	partición y offset
Retención	hasta dejar de ser necesario para recovery	según tiempo, tamaño o política de negocio
Resultado	páginas o estado local reconstruido	estado derivado, acciones y nuevos eventos

El salto conceptual puede resumirse así:

> En la clase 6, el log era evidencia interna para reconstruir un nodo. En la clase 7, el log se vuelve una historia compartida que varias aplicaciones incorporan a su propio ritmo.

El log no sustituye a la base de estado ni a la cola de trabajo en todos los casos. Es una abstracción específica que favorece retención, replay, fan-out y reconstrucción de derivados.

5 Mapa intuitivo antes del formalismo

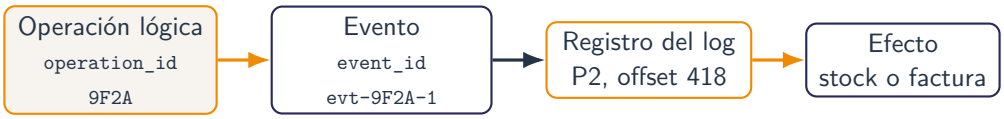
Antes de usar vocabulario de producto, conviene asociar cada concepto con una pregunta concreta:

Concepto	Imagen mental	Pregunta que abre
Log	película retenida	¿qué historia puede releerse?
Partición	carril ordenado	¿qué hechos deben compartir orden?
Offset	señalador durable	¿desde dónde reinicia el consumidor?
Grupo	equipo que reparte carriles	¿cuánto paralelismo real existe?
Idempotencia	sello de "ya aplicado"	¿qué ocurre cuando el mismo evento reaparece?
Outbox	carta guardada con el cambio	¿cómo evitar estado confirmado sin evento?
Watermark	umbral de cierre provisional	¿cuánto esperar por eventos tardíos?

La formalización posterior indicará dónde deja de servir cada analogía. Por ejemplo, un offset es un señalador dentro de una partición, pero no identifica por sí solo la operación de negocio ni demuestra que su efecto haya sido aplicado.

6 Caso conductor: CampusShop durante una venta relámpago

CampusShop confirma la operación lógica `9F2A`. Esa operación produce el evento `evt-9F2A-1`, cuyo tipo es `OrderConfirmed`. El evento se almacena, por ejemplo, en la partición 2 y recibe el offset 418.



Las tres identidades no son intercambiables:

`operation_id=9F2A`. Identifica la intención lógica original. Permite reconocer un retry del cliente como la misma compra.

`event_id=evt-9F2A-1`. Identifica el hecho publicado. Permite deduplicar su procesamiento en consumidores.

P2, offset 418. Identifica una posición en una historia física lógica concreta. El mismo número de offset puede existir en otra partición y no identifica el pedido.

Durante diez minutos, el sistema recibe cientos de miles de pedidos. Cuatro consumidores reaccionan:

- **Inventario**: descuenta stock y no puede aplicar dos veces la misma compra.
- **Facturación**: genera una factura y conserva orden por pedido.
- **Notificación**: intenta enviar un correo; puede tolerar atraso, pero debe definir si admite duplicados.

- **Analítica**: puede quedar atrasada, releer la historia y corregir agregaciones.

Si el checkout invocara sincrónicamente a los cuatro, la latencia y disponibilidad del flujo quedarían acopladas al componente más lento. Al publicar un hecho durable, el checkout espera la garantía de publicación; cada consumidor procesa después. El costo desplazado es que ahora deben diseñarse orden, progreso, redelivery, estado, tiempo y operación.

7 Vocabulario previo y acrónimos

7.1 Historia y publicación

Evento. Representación de un hecho que ya ocurrió en el dominio, con identidad, tipo, datos, tiempo y metadatos.

Registro. Unidad física lógica almacenada en una partición. En Kafka suele incluir key, value, timestamp y headers.

Productor. Aplicación que publica registros.

Broker. Servidor intermediario que recibe, almacena y sirve particiones del log.

Cluster. Conjunto de brokers que coopera para almacenar y replicar el log.

Topic. Categoría lógica de registros, dividida en una o más particiones.

Partición. Secuencia append-only ordenada de registros. Es la unidad básica de orden y paralelismo.

Key o clave. Dato utilizado para elegir partición y, con ello, qué eventos comparten orden.

7.2 Progreso y consumidores

Offset. Posición monotónica de un registro dentro de una partición.

Position. Próximo offset que una instancia viva intentará leer.

Committed offset. Punto durable desde el cual el grupo reiniciará después de una falla.

Consumer o consumidor. Aplicación que lee registros y produce estado o efectos.

Consumer group. Conjunto de consumidores que implementa una aplicación lógica y reparte particiones.

Asignación. Relación temporal entre particiones y miembros de un grupo.

Rebalance. Proceso que revoca y reasigna particiones cuando cambia la membresía o la topología.

Lag. Distancia entre el final de una partición y el progreso confirmado del consumidor.

7.3 Replicación y confirmación

ACK. *Acknowledgement*, o acuse de recibo. Indica qué condición cumplió el broker antes de responder al productor.

Líder. Réplica que acepta publicaciones y ordena registros para una partición.

Seguidor. Réplica que copia el log del líder.

Factor de replicación. Cantidad de copias configuradas para cada partición.

ISR. *In-Sync Replicas*, o réplicas consideradas suficientemente al día según la política del broker.

7.4 Semánticas y tiempo

At-most-once. Un registro produce como máximo un intento visible de efecto; puede perderse.

At-least-once. Un registro puede reentregarse; evita confirmar progreso prematuramente, pero puede repetir el efecto.

Exactly-once. Dentro de una frontera declarada, el resultado observable equivale a un solo efecto lógico pese a retries o crashes.

Idempotencia. Repetir la misma operación lógica deja el mismo estado que ejecutarla una vez.

Transacción. Frontera que vuelve visibles conjuntamente varios cambios o ninguno.

Stream. Secuencia potencialmente ilimitada de eventos.

Stateless. Cada evento puede procesarse sin recordar eventos anteriores.

Stateful. El resultado depende de historia acumulada, por ejemplo un ranking o un join.

Event time. Momento en que ocurrió el hecho en la fuente.

Ingestion time. Momento en que el sistema de streaming recibió el evento.

Processing time. Momento en que el operador procesó el evento.

Ventana. Límite temporal o por cantidad que vuelve finita una agregación.

Watermark. Señal de progreso del event time utilizada para decidir cuándo emitir o cerrar resultados.

Evento tardío. Evento cuyo timestamp queda detrás del watermark cuando llega al operador.

SLO. *Service Level Objective*, u objetivo cuantitativo de nivel de servicio.

8 Log, cola y base de estado

Las tres abstracciones pueden coexistir, pero responden preguntas distintas.

	Cola de trabajo	Log retenido	Base de estado
Objeto central	tarea pendiente	historia ordenada	valor vigente
Después de leer	puede retirarse	permanece por retención	se consulta el estado
Consumidores	compiten por tarea	mantienen progreso propio	realizan queries
Replay	no siempre natural	parte del modelo	requiere historial adicional
Uso típico	repartir trabajo	integración y streams	servir estado actual

Un log es especialmente valioso cuando:

- varios consumidores necesitan el mismo hecho;
- consumidores nuevos deben reconstruir estado;
- se requiere auditoría o replay;
- la producción no debe esperar a cada uso posterior;
- el orden por entidad es significativo.

No es una solución gratuita: la retención ocupa espacio, los contratos sobreviven a múltiples despliegues y cada consumidor debe gobernar su progreso.

9 Topics, particiones y orden

Un topic es una categoría lógica. La partición P_j puede modelarse como:

$$P_j = (e_{j,0}, e_{j,1}, \ldots, e_{j,n}).$$

El offset de $e_{j,k}$ es k. El broker agrega nuevos registros al final. La propiedad esencial es:

> El orden se define dentro de cada partición. Dos particiones no forman por sí solas una única línea de tiempo total.

Si `OrderCreated`, `PaymentAuthorized` y `OrderCancelled` deben observarse en orden para un pedido, la key natural es `order_id`. Todos los eventos de `9F2A` irán a la misma partición mientras la función de particionado y la topología sean compatibles.

9.1 La key preserva orden y distribuye carga

Una key produce dos consecuencias simultáneas:

1. define la unidad de orden;
2. concentra en la misma partición la carga de esa unidad.

Ejemplos:

Key	Orden preservado	Riesgo	Uso
`order_id`	eventos de un pedido	pedidos extremadamente activos	workflow por pedido
`student_id`	compras de un estudiante	usuario o bot caliente	límites por usuario
`campus_id`	toda una sede	partición masiva	orden global de sede excepcional
Sin key	ninguno deliberado	relaciones separadas	telemetría independiente

Una *hot partition* aparece cuando una partición recibe o procesa mucho más trabajo que las demás. El promedio del topic puede parecer sano mientras una partición domina el lag. Por eso deben medirse tasa, bytes, latencia y lag por partición.

9.2 Cantidad de particiones

Si una partición sostiene c eventos/s y el pico esperado es λ, una cota inicial es:

$$P_{min} = \left\lceil \frac{\lambda}{c} \right\rceil.$$

Con 80.000 eventos/s y 5.000 eventos/s por consumidor:

$$P_{min} = 16.$$

Con 30% de margen:

$$P_{cap} = \left\lceil \frac{80.000 \times 1{,}3}{5.000} \right\rceil = 21.$$

El cálculo es solo una base. No considera skew, tamaño de registros, compresión, costo por evento, rebalances, estado local ni límites del broker. Más particiones aumentan paralelismo, pero también metadata, conexiones, archivos, elecciones y costo de operación.

10 Publicación, réplica y ACK

Cada partición suele tener una réplica líder y seguidores. El productor publica al líder; los seguidores copian su log. El factor de replicación no define por sí solo cuándo el productor recibe éxito. Hace falta una política de ACK y un mínimo de réplicas sincronizadas.

Política conceptual	Condición antes de responder	Riesgo/costo
Sin esperar ACK	el productor no espera confirmación	desconoce si llegó
ACK del líder	el líder agregó el registro	pérdida si falla antes de replicar
ACK de réplicas requeridas	líder y réplicas sincronizadas cumplen la política	más latencia o rechazo durante degradación

La garantía depende de la combinación de:

- factor de replicación;
- réplicas que permanecen en ISR;
- configuración de ACK del productor;
- mínimo de ISR requerido;
- política de elección de líder;
- persistencia real de cada broker.

Un timeout del productor conserva la incertidumbre de clases anteriores. Puede significar que el registro no llegó, que llegó y no se replicó, o que se replicó pero la respuesta se perdió. Reintentar mejora progreso, pero puede duplicar registros.

10.1 Productor idempotente

Un productor idempotente utiliza una identidad de productor, una época y números de secuencia para que el broker reconozca el mismo lote reenviado. Esto evita ciertos duplicados producidos por retries del protocolo.

No evita:

- que dos productores distintos publiquen la misma intención lógica;
- que la aplicación genere dos `event_id`;
- que un consumidor repita un pago;
- que un sistema externo ejecute dos veces un efecto.

Por eso deben distinguirse idempotencia de transporte e idempotencia de negocio.

11 Consumidores, grupos y paralelismo

Un consumer group representa una aplicación lógica. Dentro de un grupo, cada partición se asigna a un único consumidor activo. Otro grupo puede leer la misma partición con progreso independiente.

El paralelismo activo está acotado por:

$$paralelismo_activo \leq \min(particiones, consumidores).$$

Con tres particiones y cinco consumidores, al menos dos miembros quedan ociosos. Si solo P2 está saturada, agregar consumidores no divide automáticamente ese carril. Para cambiarlo habría que revisar la unidad de particionado, el modelo de estado o el costo de esa key.

11.1 Rebalance

Cuando un miembro entra, sale o deja de renovar su sesión, el grupo reasigna particiones. Un rebalance puede implicar:

- pausa de procesamiento;
- revocación de particiones;
- confirmación o abandono de trabajo en curso;
- asignación a otro miembro;
- restauración de caché o estado local;
- calentamiento antes de recuperar throughput.

Escalar demasiado agresivamente puede crear oscilación: se agregan consumidores por lag, el rebalance reduce temporalmente la capacidad, aumenta el lag y el autoscaler agrega aún más. Se necesitan histéresis, ventanas de estabilización y métricas de tiempo útil frente a tiempo de rebalance.

12 Offset: progreso durable, no efecto durable

Deben distinguirse dos marcadores:

Position. Próximo registro que la instancia viva intentará leer.

Committed offset. Punto durable desde el que el grupo reiniciará después de una falla.

Si la posición es 101 y el committed offset es 81, el proceso ya leyó registros hasta 100, pero después de un crash puede releer desde 81. El rango 81–100 puede reaparecer.

Esto produce la pregunta más importante de la clase:

> ¿En qué orden se vuelven durables el efecto de negocio y el progreso del consumidor?

13 Semánticas de procesamiento mediante timelines

13.1 At-most-once: progreso antes que efecto

1. el consumidor lee el offset 42;
2. confirma que reiniciará en 43;

3. ocurre un crash;
4. el efecto de inventario nunca se ejecuta;
5. al reiniciar, 42 no reaparece.

Resultado: el evento produce como máximo un efecto, pero puede perderse. Puede ser aceptable para una métrica aproximada, no para descontar la última unidad de stock.

13.2 At-least-once: efecto antes que progreso

1. el consumidor lee 42;
2. descuenta stock;
3. ocurre un crash antes de confirmar 43;
4. al reiniciar, 42 reaparece;
5. el descuento puede repetirse.

At-least-once permite reintentar procesamiento sin adelantar el progreso, pero no garantiza por sí solo que el efecto termine con éxito ni que ocurra una sola vez.

13.3 Inbox idempotente

Para un efecto local en una base transaccional, una solución robusta es guardar juntos:

- el `event_id` procesado;
- la modificación de negocio;
- opcionalmente el resultado que debe devolverse ante replay.

Ejemplo:

```
BEGIN;
INSERT INTO inbox(event_id) VALUES ('evt-9F2A-1');
UPDATE inventory SET stock = stock - 1 WHERE product_id = 17;
COMMIT;
```

La restricción única sobre `event_id` convierte un replay en no-op o en recuperación del resultado anterior. La atomicidad es esencial: si el efecto y la marca se persisten por separado, un crash entre ambas operaciones reabre la ventana de duplicación.

14 Entrega, procesamiento y efecto de negocio

La palabra "procesado" puede referirse a capas diferentes:

Capa	Pregunta	Ejemplo
Entrega	¿cuántas veces llegó el registro?	dos polls del offset 42
Estado interno	¿cuántas veces cambió el state store?	contador incrementado una vez
Efecto de negocio	¿cuántas veces ocurrió afuera?	pago, email o stock

Un pipeline puede mantener estado interno exactamente una vez y enviar dos correos. También puede recibir dos veces un registro, deduplicarlo y producir un solo efecto lógico. La semántica debe formularse como:

$$(\text{objeto},\ \text{frontera},\ \text{fallas cubiertas}).$$

15 Exactly-once dentro de una frontera

En un pipeline Kafka–Kafka, una transacción puede coordinar:

- offsets de entrada;
- cambios de estado y su changelog;
- registros de salida.

Si ocurre un crash antes del commit, la transacción se aborta, los outputs no se vuelven visibles a consumidores configurados para leer datos confirmados y el input se reprocesa. Si el commit termina, estado, output y progreso se observan conjuntamente.

La garantía termina donde termina la transacción. Un proveedor de correo, una API bancaria o una base externa no participan automáticamente. Para esos efectos se necesitan protocolos específicos, claves idempotentes, inbox/outbox o reconciliación.

> La frase "Kafka garantiza exactly-once" está incompleta. Debe indicar exactamente una vez qué resultado, dentro de qué frontera y ante qué fallas.

16 Dual write y transactional outbox

Un dual write aparece cuando un servicio modifica su base y publica un evento en un broker independiente.

16.1 Base primero

Si la base confirma y el proceso cae antes de publicar, existe estado sin evento.

16.2 Evento primero

Si el evento se publica y la base aborta o el proceso cae antes del commit, existe evento sobre un estado que nunca confirmó.

No existe un orden universal que vuelva atómicas dos escrituras independientes.

16.3 Outbox

Transactional outbox guarda el cambio de negocio y el evento pendiente dentro de la misma transacción local:

```
BEGIN;
INSERT INTO orders(order_id, status) VALUES ('9F2A', 'CONFIRMED');
INSERT INTO outbox(event_id, aggregate_id, event_type, payload)
VALUES ('evt-9F2A-1', '9F2A', 'OrderConfirmed', '{...}');
```

```
COMMIT;
```

Un *relay* lee la outbox y publica. Otra alternativa es CDC, *Change Data Capture* o captura de cambios, que observa el log transaccional y transforma cambios en eventos.

El relay puede publicar, perder el ACK y volver a publicar. Por eso outbox garantiza que la intención de publicar no se pierda junto al estado local, pero conserva redelivery. El consumidor necesita inbox o idempotencia.

Un pipeline sólido es:

$$\text{transacción de negocio} + \text{outbox} \to \text{relay repetible} \to \text{log} \to \text{inbox} + \text{efecto local}.$$

Cada flecha tiene métricas y fallas propias.

17 Procesamiento stateless y stateful

Un operador stateless transforma cada evento de forma independiente. Ejemplos: validar formato, proyectar campos o filtrar por tipo.

Un operador stateful conserva historia. Ejemplos:

- conteo por usuario;
- ranking por ventana;
- join entre pedidos y pagos;
- detección de secuencias de fraude;
- deduplicación por event ID.

El estado local acelera procesamiento, pero debe recuperarse de manera compatible con el progreso. Un changelog registra cambios del state store; un checkpoint o snapshot reduce el tiempo de restauración. Si offset y estado se restauran desde momentos incompatibles, pueden aparecer omisiones o duplicaciones.

18 Tiempo en streams

En sistemas batch, el conjunto de datos tiene un final. En un stream, la entrada puede no terminar. Para responder preguntas como “ventas por cinco minutos” se necesita una ventana.

18.1 Tres tiempos

Event time. Hora en que ocurrió el hecho según la fuente autorizada.

Ingestion time. Hora en que el pipeline recibió el evento.

Processing time. Hora en que el operador lo procesó.

Un teléfono puede permanecer sin red y enviar una jugada siete minutos después. Processing time la ubicaría en el momento de llegada; event time intenta conservar el momento del hecho.

18.2 Ventanas

Una ventana tumbling de cinco minutos divide el tiempo en intervalos disjuntos. Una ventana sliding puede solaparse y producir una salida cada paso. La ventana debe declararse junto con key, reloj, política de cierre y estado retenido.

18.3 Watermarks

Un watermark es una señal operacional de progreso del event time. No afirma que ningún evento anterior llegará jamás. Expresa que el sistema decide emitir o cerrar resultados bajo una tolerancia de retraso.

Watermark largo:

- mayor completitud esperada;
- más estado retenido;
- más latencia antes del resultado final.

Watermark corto:

- menor latencia;
- más correcciones, descarte o resultados incompletos.

18.4 Evento tardío trabajado

E3 ocurrió a 10:04:50, llegó a 10:07:10 y pertenece por event time a 10:00–10:05. Llegó 2 min 10 s después del cierre nominal de la ventana. Según la política puede:

- descartarse y contabilizarse;
- enviarse a un flujo de eventos tardíos;
- corregir una salida anterior;
- reabrir la ventana;
- activar una revisión especial.

La opción depende del costo de esperar y del costo de una respuesta incompleta.

19 Caso complementario: ranking de Asteria Online

El servidor autoritativo de combate emite `CombatFinished`. El ranking en vivo se calcula por ventanas de cinco minutos.

- **Identidad**: `combat_id`, no offset.
- **Key**: `player_id`, si se requiere orden por jugador.
- **Tiempo**: event time del servidor autoritativo, no del cliente manipulable.
- **Semántica**: at-least-once con deduplicación por `combat_id`.
- **SLO**: 99% de resultados visibles en menos de 30 s.
- **Late data**: el ranking visible puede corregirse; la entrega de premios exige una etapa de cierre más fuerte.

El mismo stream puede alimentar una vista eventual para la experiencia del jugador y un proceso más controlado para recompensas. No se exige una única garantía para todos los usos.

20 Operación: métricas que conectan historia, progreso y efecto

Un pipeline no está sano solo porque el broker responde. Deben observarse al menos:

- tasa de producción y consumo por partición;
- lag de offset por partición;
- edad del evento más antiguo pendiente;
- latencia extremo a extremo hasta el efecto útil;
- retries, errores y throttling del productor;
- rebalances por hora y tiempo detenido;
- tiempo de restauración de state stores;
- tamaño y edad de outbox e inbox;
- duplicados detectados;
- eventos tardíos y correcciones;
- eventos en cuarentena.

Una DLQ, *Dead-Letter Queue* o cola de mensajes no procesables, evita que un evento permanentemente inválido bloquee toda una partición. No debe convertirse en un cementerio silencioso: requiere propietario, razón, alerta, retención y procedimiento de replay.

20.1 El mismo lag puede tener causas distintas

1. **Ingreso mayor que capacidad**: todas las particiones crecen.
2. **Hot partition**: una partición concentra keys o eventos costosos.
3. **Dependencia lenta**: el consumidor espera una base o API externa.
4. **Rebalances**: el grupo pierde tiempo útil reasignando y restaurando.
5. **Poison pill**: un evento bloquea retries sin progreso.

La intervención correcta depende de la causa. Agregar consumidores no resuelve automáticamente una hot partition, una dependencia lenta ni un evento no procesable.

21 Ejemplos cuantitativos trabajados

21.1 Del lag en offsets al tiempo de atraso

Suponga que una partición termina en el offset 2.000.000 y el grupo confirmó hasta 1.400.000. El lag es:

$$L = 2.000.000 - 1.400.000 = 600.000 \text{ registros.}$$

Ese número no indica por sí solo cuánto tiempo lleva ponerse al día. Si llegan 30.000 eventos/s y el consumidor procesa 50.000 eventos/s, la capacidad neta de recuperación es:

$$\mu - \lambda = 50.000 - 30.000 = 20.000 \text{ eventos/s}.$$

Bajo carga estable y sin nuevas pausas, el tiempo ideal de recuperación es:

$$T_{catchup} = \frac{600.000}{20.000} = 30 \text{ s}.$$

Si la tasa de consumo es menor o igual a la tasa de ingreso, el backlog no puede reducirse. Este cálculo explica por qué conviene medir simultáneamente offsets, tasas y edad del evento más antiguo.

21.2 Costo de restaurar estado después de un rebalance

Un operador de ranking mantiene un state store de 24 GB. Durante la restauración puede leer el changelog a 200 MB/s efectivos. Ignorando calentamiento y contención:

$$T_{restore} \approx \frac{24 \times 1024 \text{ MB}}{200 \text{ MB/s}} \approx 123 \text{ s}.$$

Un autoscaler que agregue y retire miembros cada minuto nunca permitiría que el grupo alcance régimen estable. El tamaño de estado, el throughput de restore y la frecuencia de rebalances deben formar parte del diseño de capacidad.

21.3 Contrato mínimo de un evento

Un evento durable debería poder interpretarse meses después y distinguirse de cualquier replay:

```
{
  "event_id": "evt-9F2A-1",
  "aggregate_id": "9F2A",
  "event_type": "OrderConfirmed",
  "schema_version": 2,
  "occurred_at": "2026-09-10T12:43:18Z",
  "producer": "campusshop-checkout",
  "trace_id": "tr-77A1",
  "payload": {"student_id": 8421, "product_id": 17}
}
```

Cada campo cumple una función distinta:

- `event_id` soporta deduplicación;
- `aggregate_id` agrupa la historia de una entidad;
- `schema_version` permite evolución explícita;
- `occurred_at` representa event time;
- `trace_id` ayuda a reconstruir causalidad operacional.

El payload debe contener información suficiente para el contrato, pero no datos sensibles innecesarios.

21.4 Frontera de confirmación del productor

Considere una partición con líder A y seguidores B y C. Si A responde después de agregar localmente pero antes de que B o C copien el registro, una falla de A puede perder una publicación ya reconocida. Si el productor espera una política que exija más réplicas sincronizadas, reduce esa ventana, pero puede quedar sin capacidad de publicar cuando el conjunto requerido no está disponible.

La decisión debe expresarse como una promesa observable, no como un valor aislado de configuración: "confirmamos cuando el registro fue aceptado por un conjunto que tolera la pérdida de una réplica" o "priorizamos disponibilidad del productor y admitimos una ventana de pérdida ante failover".

22 Método reusable de diseño

Para diseñar un pipeline:

1. nombrar el hecho y su identidad estable;
2. declarar la unidad de orden;
3. elegir key y cantidad de particiones;
4. definir la condición de ACK y pérdida tolerable;
5. definir el efecto de cada consumidor;
6. ubicar el commit de progreso;
7. diseñar idempotencia o transacción;
8. localizar dual writes y efectos externos;
9. elegir reloj, ventana y política de late data;
10. definir SLO, métricas y pruebas de falla.

Aplicado al marco del curso:

$$D = (P, F, G, S, T).$$

Para CampusShop:

- P: propagar pedidos a consumidores independientes;
- F: timeout, crash, replay, rebalance, skew y evento tardío;
- G: no perder pedidos, efecto idempotente y actualización dentro del SLO;
- S: log particionado, réplica, offsets, inbox/outbox, estado y ventanas;
- T: almacenamiento, complejidad operacional, redelivery, latencia y reparación.

23 Pruebas de falla recomendadas

Una afirmación de garantía debe acompañarse con un experimento reproducible:

1. publicar eventos con IDs conocidos;
2. matar al productor después de enviar y antes de recibir ACK;

3. matar al consumidor después del efecto y antes del commit del offset;
4. agregar o retirar miembros para forzar rebalance;
5. introducir una key caliente;
6. retrasar selectivamente eventos para probar watermarks;
7. verificar pérdida, duplicación lógica, orden, tiempo de restauración y SLO.

El criterio de éxito no es "el cluster volvió". Es que la historia, el progreso y el efecto final satisfagan el contrato declarado.

24 Errores conceptuales frecuentes

- afirmar que un topic tiene orden global;
- usar offset como identidad de negocio;
- confundir position con committed offset;
- describir at-least-once como garantía de éxito del negocio;
- decir exactly-once sin objeto ni frontera;
- creer que producer idempotence vuelve idempotente al consumidor;
- proponer outbox sin inbox o deduplicación;
- medir solo lag promedio;
- agregar consumidores cuando el límite es una sola partición;
- ignorar el costo de rebalance y restore;
- usar processing time cuando el dominio necesita event time;
- cerrar ventanas sin política para eventos tardíos;
- tratar DLQ como destino final sin operación.

25 Glosario operativo

Broker. Servidor que almacena y sirve particiones.

Topic. Nombre lógico de una colección de registros.

Partición. Secuencia ordenada append-only dentro de un topic.

Key. Dato que dirige eventos relacionados a una partición.

Offset. Posición de un registro dentro de una partición.

Position. Próximo offset que una instancia intentará leer.

Committed offset. Progreso durable del grupo.

Lag. Distancia o edad entre el final del log y el progreso confirmado.

Rebalance. Reasignación de particiones entre miembros.

State store. Estado materializado por un procesador stateful.

Changelog. Log de cambios utilizado para reconstruir un state store.

Outbox. Registro transaccional de eventos pendientes junto al cambio de negocio.

Inbox. Registro transaccional de IDs procesados junto al efecto local.

CDC. Captura de cambios de una base o su log transaccional.

Watermark. Señal de progreso asumido del event time.

DLQ. Cuarentena operada para eventos que no pudieron procesarse.

26 Ruta esencial y profundización

Para dominar la clase, priorice esta ruta esencial:

1. registro, append, log y progreso durable;
2. partición, key y orden local;
3. consumidor, position y committed offset;
4. timelines de at-most-once y at-least-once;
5. event ID, idempotencia e inbox;
6. dual write y transactional outbox;
7. lag por partición y evidencia operacional.

Los siguientes temas son profundización y deben estudiarse después de dominar los timelines básicos:

- producer idempotence e ISR;
- transacciones Kafka–Kafka;
- state store y changelog;
- watermarks y políticas avanzadas de late data;
- poison events y DLQ.

La prioridad no es memorizar configuraciones. La prioridad es poder dibujar una ejecución adversarial, localizar qué estado sobrevivió y justificar la recuperación.

27 Bibliografía guiada

- Martin Kleppmann, *Designing Data-Intensive Applications*, capítulos 11 y 12: logs, procesamiento de streams y sistemas derivados.
- Apache Kafka, documentación oficial: diseño, producers, consumers, delivery semantics y transactions.
- Apache Flink, documentación de event time y watermarks: tratamiento de tiempo y eventos tardíos.
- Debezium, documentación del patrón outbox: identidad del evento y captura de cambios.
- Kreps, Narkhede y Rao, *Kafka: a Distributed Messaging System for Log Processing*: motivación y diseño original.

Ruta sugerida: primero leer DDIA para el modelo conceptual; después verificar detalles de versión en la documentación oficial; finalmente ejecutar los timelines y demos del paquete.

28 Conexión con el Trabajo Final Integrador

Cada equipo debería entregar para un flujo:

- hecho, productor y owner;
- event ID y aggregate ID;
- topic, key, particiones y orden;
- consumidores y semántica por consumidor;
- estrategia de publicación desde la base;
- frontera de exactly-once si existe;
- efecto externo fuera de la frontera;
- reloj, ventana y late-data policy;
- SLO extremo a extremo;
- prueba de crash, replay y rebalance.

No alcanza con escribir "usaremos Kafka porque escala". Debe demostrarse qué problema resuelve el log y qué costo nuevo introduce.

29 Autoevaluación

1. ¿Qué diferencia existe entre WAL local y log distribuido?
2. ¿Por qué un log no es simplemente una cola?
3. ¿Qué campos identifican operación, evento y posición?
4. ¿Qué orden garantiza una partición?
5. ¿Cómo puede una key producir una hot partition?
6. ¿Por qué más particiones no son gratuitas?
7. ¿Qué función cumplen líder, seguidores e ISR?
8. ¿Qué significa cada política de ACK?
9. ¿Qué duplicados evita un productor idempotente?
10. ¿Qué limita el paralelismo de un consumer group?
11. ¿Qué diferencia hay entre position y committed offset?
12. ¿Cómo aparece pérdida en at-most-once?
13. ¿Cómo aparece duplicación en at-least-once?
14. ¿Qué debe persistirse atómicamente en una inbox?
15. ¿Qué tres capas se esconden detrás de "procesado"?

16. ¿Qué puede incluir una transacción Kafka–Kafka?
17. ¿Por qué un email queda fuera de esa frontera?
18. ¿Qué ventana de falla resuelve outbox?
19. ¿Por qué el relay puede publicar dos veces?
20. ¿Qué debe recuperar un operador stateful?
21. ¿Cuál es la diferencia entre event, ingestion y processing time?
22. ¿Qué trade-off expresa un watermark?
23. ¿Por qué lag cero no demuestra corrección?
24. ¿Qué evidencia demostraría que el pipeline cumple su SLO?

30 Síntesis final

El log conserva una historia durable; la partición define el orden disponible; el offset registra progreso; la transacción coordina visibilidad dentro de una frontera; la idempotencia protege el efecto; y el modelo temporal decide cuándo un resultado continuo es suficientemente completo. Diseñar streaming consiste en declarar y medir cada una de esas fronteras.